Dynamic memory allocation for over-aligned data

Doc. No.:	P0035R1
Revises:	N3396
Date:	2015-12-29
Reply to:	Clark Nelson
Phone:	+1-503-712-8433
Email:	clark.nelson@intel.com
Audience:	Core, Library Evolution

Problem statement

To codify widespread existing practice, C++11 added the ability to specify increased alignment (a.k.a. over-alignment) for class types. Unfortunately (but also consistently with existing practice), C++11 did not specify any mechanism by which over-aligned data can be dynamically allocated correctly (i.e. respecting the alignment of the data). For example:

```
class alignas(16) float4 {
        float f[4];
};
float4 *p = new float4[1000];
```

In this example, not only is an implementation of C++ **not** required to allocate properly-aligned memory for the array, for practical purposes it is very nearly required to do the allocation incorrectly. In any event, it is certainly required to perform the allocation by a process that does **not** take the specified alignment value into account.

This represents a hole in the support for alignment in the language, which really needs to be filled.

History

With the exception of new paragraphs, which are boxed, and a few updates taking into account recent WD changes (mostly sized deallocation), this document is substantially the same as N3396, which was discussed by EWG at the 2012 Portland meeting.

Since that time, Intel has released a compiler that largely implements the language changes discussed herein, except that, to guarantee backward compatibility, the additional overloads are declared in a new header (`<aligned_new>`), instead of being predeclared or declared in `<new>`.

To date, there has not yet been enough experience with the implementation to prove its viability. Nevertheless, it seems appropriate to get this issue back on the committee's radar, so that a decision can be made about it for the C++17 time frame.

Acknowledgements

Pablo Halpern has provided me with much valuable feedback and assistance.

Design considerations

Backward compatibility

One of the first questions that needs to be settled about the future direction is the degree to which backward compatibility with C++11/14 needs to be maintained. On the one hand, in an ideal world, for an example like the one above, it would be obvious that the specified alignment should be honored.

On the other hand, there's no way to achieve that ideal without at least potentially changing the behavior of some programs conforming to an earlier standard. For example, a program might asssume control of dynamic allocation through the use of class-specific `operator new` and `operator delete` functions, or by replacing the global functions. These functions don't take any alignment argument. If a different function is used instead, which is somehow passed an alignment value, some degree of backward compatibility is lost.

When backward compatibility and the ideal future direction are in conflict, which should take precedence, and to what degree?

If perfect backward compatibility were required, one way to ensure that might be to require that a new header — say `<aligned_new>` — be included in order to get new dynamic allocation for over-aligned types. But that would sacrifice convenience and/or correctness; using `alignas` by itself would presumably never be enough to get correctly aligned dynamic allocation.

Another obvious position to take would be that backward compatibility with C++98, which had no alignment specifier, needs to

be complete. This might suggest that dynamic allocation should differ between types involving alignment specifiers and types that don't — which some might consider to be an unfortunate complication.

In C++11/14, when an over-aligned class type has its own dynamic memory allocation functions, it would be reasonable to hope that those functions already do the right thing with respect to alignment, and dangerous to make any change. However, the only way over-alignment could be accommodated by global allocation and deallocation functions would be to replace them with functions that always provide the strictest alignment used by any type in the program. It may be reasonable to assume that very few programs go to that length, instead of using class-specific allocation/deallocation.

Therefore, it may be acceptable to abandon backward compatibility with C++14 with respect to calling a global allocation function for dynamic allocation of an over-aligned type. But if so, that may well be the only acceptable case.

Passing the alignment value

To minimize the possibility of conflict with existing placement allocation functions, it might be advisable to invent a new standard enumeration type to use for alignment parameters; for example:

```
namespace std {
        enum class align_val_t: size_t;
};
void *operator new(std::size_t, std::align_val_t);      // new overload
```

It's not clear that this type would need any named constants of its own; it just needs to be able to represent alignment values, which are associated with type `size_t`. It should perhaps nevertheless be a scoped enumeration, to prevent the possibility that a value of that type would inadvertently be converted to some integer type, and match an existing placement allocation function.

If an allocation function that takes an alignment value is available, it should be used, for the sake of generality; but if no such function is available, a function that doesn't take one should be used, for backward compatibility. This suggests a new rule for new-expressions: attempting to find an allocation function in two phases, with two different sets of arguments.

Class-specific allocation and deallocation

It should be kept in mind that, under the current language rules, any class-specific allocation functions effectively hide all global allocation functions, including the ones in the standard library. For example, the following is invalid:

```
#include <new>
class X {
        void *operator new(size_t);
        // no operator new(size_t, std::nothrow_t&)
        void operator delete(void *);
};
X *p1 = new X;          // uses X::operator new(size_t)
X *p2 = new(nothrow) X; // error
// ::operator new(size_t, std::nothrow_t&) is not considered
```

It is possible to imagine adjusting the rules to enable finding an alignment-aware allocation function more often, but that would also make it more likely that some programmers would write programs believing — incorrectly — that they have taken over complete control of the way that their class is dynamically allocated.

Unified vs. distinct arenas

What implementation techniques should the standard allow for allocation and deallocation of aligned memory?

In POSIX, there is a function named `posix_memalign` that can allocate over-aligned memory; `free` is used to free the blocks it allocates.

On Windows, on the other hand, of course `malloc`, `realloc` and `free` are supported for default-aligned memory. In addition, for over-aligned memory, there are functions named `_aligned_malloc`, `_aligned_realloc`, and `_aligned_free`. Memory that's allocated by `_aligned_malloc` must be freed by `_aligned_free`, and memory that's allocated by `malloc` must be freed by `free`. So logically, there are two disjoint, non-interoperable memory arenas; the program has to know to which arena a block belongs (i.e. how it was allocated) in order to be able to free it.

This is almost certain to be true of any implementation where over-aligned memory allocation is layered on top of “plain old” default-aligned memory allocation. There are probably many such implementations, and they're not likely to go away soon.

In an environment where information about the method used to allocate a block of memory can be lost, having distinct arenas (i.e. distinct deallocation functions) could be inconvenient. A program whose operation depends on the assumption that `operator new` is equivalent to `malloc` is effectively an environment where information about the method used to allocate a block of memory

is lost.

But in a well-written, portable C++ program, at the point where memory is deallocated, the type of the object being deleted — and therefore whether it is over-aligned — is known. This knowledge could, and probably should, be used to support layered implementations of over-aligned memory allocation.

This implies that, just as a new-expression for an over-aligned type should look for an alignment-aware allocation function, so should a delete-expression for a pointer to an over-aligned type look for an alignment-aware deallocation function. Presumably this would be done by selecting a deallocation function to which the alignment value can be passed, even though probably very few implementations will actually have any use for that value.

Nitty-gritty

For exactly what classes should the allocation method change? Plausible answers include:

1. Those affected by an `alignas` that actually specifies over-alignment.
2. Those affected by an explicit `alignas`, even if the alignment value is basic (i.e. small).

The first answer seems to be right from a pragmatic perspective, but one consequence is that the behavior of a program might depend (in a new way) on an implementation-defined parameter. But if the only difference between alignment-aware and alignment-unaware allocation/deallocation functions is the actual allocation mechanism (i.e., in a well-designed program), this should not be a problem. It's rather like the implementation's license to elide certain copies, which implies that a copy constructor had really better just make a copy.

The below WD changes use the first answer, through use of “over-aligned”. The Intel implementation uses the first answer by default, but has a command-line option to select the second answer, for the sake of experimentation.

Also, it should be noted that the over-alignment threshold used by the Intel implementation doesn't exactly match the standard's definition of basic alignment. The threshold used is actually the alignment observed to be guaranteed by the implementation of `malloc` for the target environment. (In all of the environments tested, this turned out to be twice the size of a pointer.)

Assuming the existence of a variety of allocation functions, which one should be used for an over-aligned allocation? I believe the answer should be the first one from the following list that is known to exist:

1. class-specific and alignment-aware
2. class-specific and alignment-unaware
3. global and alignment-aware
4. [global and alignment-unaware]

It makes sense for a class-specific, alignment-unaware allocation function to be preferred over one that is global and alignment-aware, because there are many cases where a class-specific allocation function has enough information, even without an explicit parameter, to do the allocation with sufficient alignment. (Likely exceptions include a template class with a base or member of a type that is a template parameter, and a derived class that inherits its allocation function from a base class, and also adds a member or base of over-aligned type.)

If a global, alignment-aware allocation function is predeclared, then it will never be necessary to use a global, alignment-unaware allocation function for an over-aligned type; hence the brackets around item 4.

It should be noted that an alignment-aware allocation function would be perfectly capable of performing an alignment that would suffice for an alignment-unaware function. In other words, from some perspective, it would make sense to let `operator new(size_t)` call `operator new(size_t, align_val_t)`, filling in the alignment value that it feels it needs to satisfy, and move the allocation loop to the alignment-aware function. But that would be pretty novel, so I have chosen not to propose it.

Outline of possible working paper changes

The following changes are believed to be substantially complete. The particularly important changes are presented first.

Mainly for simplicity, here I suggest that the new overloads should be added to `<new>`, and for consistency with that, that they should also be predeclared. But if 100% backward compatibility with C++14 is considered necessary, then the new overloads probably need to be declared in a new library header (possibly `<aligned_new>`). It's also possible to imagine requiring the declarations be in a new header, but making it implementation-defined whether that header is included by `<new>`, perhaps with

the expectation that the actual choice will be left to users, under the control of a command-line option or macro setting.

There is one change of terminology worth noting. Today, the phrase “placement new” is ambiguous. In some contexts it means adding arguments to a call to an allocation function, with any types and unspecified purpose. In other contexts, it is used to refer specifically to cases where there is a single additional argument of type `void *`, in which case the allocation function doesn't actually allocate anything. I refer to the latter cases as “non-allocating”, and refer to “allocating” cases to distinguish them when necessary.

Principal changes

These introduce the required new library functions, and specify the language rules that trigger their use.

Change 18.6, header `<new>` synopsis:

```
namespace std {
        class bad_alloc;
        class bad_array_new_length;
        enum class align_val_t: size_t;
        struct nothrow_t {};
        extern const nothrow_t nothrow;
        typedef void (*new_handler)();
        new_handler get_new_handler() noexcept;
        new_handler set_new_handler(new_handler new_p) noexcept;
};
void* operator new(std::size_t size);
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void* operator new[](std::size_t size);
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;

void* operator new(std::size_t size, std::align_val_t alignment);
void* operator new(std::size_t size, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::align_val_t alignment,
                   std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment,
                   std::size_t size) noexcept;
void* operator new[](std::size_t size, std::align_val_t alignment);
void* operator new[](std::size_t size, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment,
                   const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment,
                   std::size_t size) noexcept;

void* operator new  (std::size_t size, void* ptr) noexcept;
void* operator new[](std::size_t size, void* ptr) noexcept;
void operator delete  (void* ptr, void*) noexcept;
void operator delete[](void* ptr, void*) noexcept;
```

Change 5.3.4p13:

> The *new-placement* syntax ~~is~~ may be used to supply additional arguments to an allocation function. ~~If used, overload~~ Overload resolution is performed on a function call created by assembling an argument list. ~~consisting of~~ The first argument is the amount of space requested ~~(the first argument)~~ , and has type `std::size_t`. If the type of the allocated object is over-aligned, the next argument is the type's alignment, and has type `std::align_val_t`. ~~and the~~ If the *new-placement* syntax is used, its expressions ~~in the *new-placement* part of the *new-expression* (~~ are the ~~second and~~ succeeding arguments~~)~~. ~~The first of these arguments has type `std::size_t` and the remaining arguments have the corresponding types of the expressions in the *new-placement*.~~ If no matching function is found and the allocated object type is over-aligned, the alignment argument is removed from the argument list, and overload resolution is performed again.

Change 5.3.4p14:

> [*Example:*
>
> - `new T` results in a call of either `operator new(sizeof(T))` or `operator new(sizeof(T), static_cast<std::align_val_t>`

`(alignof(T))`,

- `new(2,f) T` results in a call of either `operator new(sizeof(T),2,f)` or `operator new(sizeof(T), static_cast<std::align_val_t>(alignof(T)),2,f)`,
- `new T[5]` results in a call of either `operator new[](sizeof(T)*5+x)` or `operator new[](sizeof(T)*5+x, static_cast<std::align_val_t>(alignof(T)))`, and
- `new(2,f) T[5]` results in a call of either `operator new[](sizeof(T)*5+y,2,f)` or `operator new[](sizeof(T)*5+y, static_cast<std::align_val_t>(alignof(T)),2,f)`.

...

Change 3.7.4.2p2:

> Each deallocation function shall return `void` and its first parameter shall be `void*`. A deallocation function ~~can~~ may have more than one parameter. ~~The global `operator delete` with exactly one parameter is a usual (nonplacement) deallocation function. The global `operator delete` with exactly two parameters, the second of which has type `std::size_t`, is a usual deallocation function. Similarly, the global `operator delete[]` with exactly one parameter is a usual deallocation function. The global `operator delete[]` with exactly two parameters, the second of which has type `std::size_t`, is a usual deallocation function.[37] If a class `T` has a member deallocation function named `operator delete` with exactly one parameter, then that function is a usual deallocation function. If class `T` does not declare such an `operator delete` but does declare a member deallocation function named `operator delete` with exactly two parameters, the second of which has type `std::size_t`, then this function is a usual deallocation function. Similarly, if a class `T` has a member deallocation function named `operator delete[]` with exactly one parameter, then that function is a usual (non-placement) deallocation function. If class `T` does not declare such an `operator delete[]` but does declare a member deallocation function named `operator delete[]` with exactly two parameters, the second of which has type `std::size_t`, then this function is a usual deallocation function.~~ A *usual deallocation function* is a deallocation function that has:
>
> - exactly one parameter; or
> - exactly two parameters, the type of the second being either `std::align_val_t` or `std::size_t`[37]; or
> - exactly three parameters, the type of the second being `std::align_val_t` and the type of the third being `std::size_t`.
>
> *Footnote:* 37) ~~This deallocation function~~ The global `operator delete[](void*, std::size_t)` precludes use of an allocation function `void operator new(std::size_t, std::size_t)` as a placement allocation function (C.3.2).
>
> A deallocation function ~~can~~ may be an instance of a function template. Neither the first parameter nor the return type shall depend on a template parameter. [*Note:* That is, a deallocation function template shall have a first parameter of type `void*` and a return type of `void` (as specified above). —*end note*] A deallocation function template shall have two or more function parameters. A template instance is never a usual deallocation function, regardless of its signature.

Adding the new overloads to the set of "usual" deallocation functions in the same style as the previous formulation would have required outrageous verbosity. My new formulation is much more concise and (to my eyes) comprehensible, but there is a technical difference worth noting.

Previously, in class scope, if deallocation functions with and without a size parameter are both declared, the one with the size parameter was not "usual". My simpler formulation includes both overloads. But it is not difficult to tweak the selection algorithm to produce the same result as previously.

Change 5.3.5p10:

> If deallocation function lookup finds ~~both a~~ more than one usual deallocation function ~~with only a pointer parameter and a usual deallocation function with both a pointer parameter and a size parameter~~, the function to be called is selected as follows:
>
> - If the type is over-aligned, a function with an alignment parameter is preferred; otherwise a function with no alignment parameter is preferred. If exactly one preferred function is available, that function is selected and the selection process terminates. If more than one preferred function is available, all non-preferred functions are eliminated from further consideration.
> - If the deallocation functions have class scope, the one without a size parameter is selected.
> - If the type is complete and if, for the second alternative (delete array) only, the operand is a pointer to a class type with a non-trivial destructor or a (possibly multi-dimensional) array thereof, the function with ~~two parameters~~ a size parameter is selected.
> - Otherwise, it is unspecified ~~which of the two deallocation functions~~ whether a deallocation function with a

size parameter is selected.

ISSUE: What if a class has a trivial destructor and only one usual deallocation function, taking a size? What value is allowed (or required) to be passed as the size argument when an array of such type is deleted? Should such a class be ill-formed?

Change 5.3.5p11:

> When a *delete-expression* is executed, the selected deallocation function shall be called with the address of the block of storage to be reclaimed as its first argument. If a deallocation function with an alignment parameter is used, the alignment of the type is passed as the corresponding argument. ~~and (if the two-parameter~~ If a deallocation function with a size parameter is used~~)~~, the size of the block is passed as ~~its second~~ the corresponding argument.[82]

Consequent changes

Most of these changes are just reflecting the implications of the above changes through the rest of the document.

If the new overloads should be predeclared, change 3.7.4p2:

> The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable (18.6.1). A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library (17.6.4.6). The following allocation and deallocation functions (18.6) are implicitly declared in global scope in each translation unit of a program.
>
> ```
> void* operator new(std::size_t);
> void* operator new[](std::size_t);
> void operator delete(void*);
> void operator delete[](void*);
> void operator delete(void*, std::size_t);
> void operator delete[](void*, std::size_t);
> ```
>
> ```
> void* operator new(std::size_t, std::align_val_t);
> void* operator new[](std::size_t, std::align_val_t);
> void operator delete(void*, std::align_val_t);
> void operator delete[](void*, std::align_val_t);
> void operator delete(void*, std::align_val_t, std::size_t);
> void operator delete[](void*, std::align_val_t, std::size_t);
> ```
>
> These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`. [*Note:* The implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression* or function call that refers to one of these functions without including the header `<new>` is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` is ill-formed unless the name has been declared by including the appropriate header. —*end note*] Allocation and/or deallocation functions ~~can~~ may also be declared and defined for any class (12.5).

Change 3.7.4p1:

> Objects can be created dynamically during program execution (1.9), using *new-expressions* (5.3.4), and destroyed using *delete-expressions* (5.3.5). A C++ implementation provides access to, and management of, dynamic storage via the global *allocation functions* `operator new` and `operator new[]` and the global *deallocation functions* `operator delete` and `operator delete[]`. [*Note:* The non-allocating forms described in [new.delete.placement] are, semantically speaking, not allocation or deallocation functions. —*end note*]

This is intended as a clarification of what seems already to be implied by [new.delete.placement]:

> The provisions of (3.7.4) do not apply to these reserved placement forms of `operator new` and `operator delete`.

Change 3.7.4.2p3:

> If a deallocation function terminates by throwing an exception, the behavior is undefined. The value of the first argument supplied to a deallocation function may be a null pointer value; if so, and if the deallocation function is one supplied in the standard library, the call has no effect. ~~Otherwise, the behavior is undefined if the value supplied to `operator delete(void*)` in the standard library is not one of the values returned by a previous invocation of either `operator new(std::size_t)` or `operator new(std::size_t, const std::nothrow_t&)` in the standard library, and the behavior is undefined if the value supplied to `operator delete[](void*)` in the standard library is not one of the values~~

~~returned by a previous invocation of either `operator new[](std::size_t)` or `operator new[](std::size_t, const std::nothrow_t&)` in the standard library.~~

These requirements apply only to the library implementations, and are already stated in 18.6.

Change 3.7.4.3p2:

> A pointer value is a *safely-derived pointer* to a dynamic object only if it has an object pointer type and it is one of the following:
>
> - the value returned by a call to the C++ standard library implementation of `::operator new(std::size_t)` or `::operator new(std::size_t, std::align_val_t)`;[37]
> - ...

Change 5.3.4p1:

> The *new-expression* attempts to create an object of the *type-id* (8.1) or *new-type-id* to which it is applied. The type of that object is the *allocated type*. This type shall be a complete object type, but not an abstract class type or array thereof (1.8, 3.9, 10.4). ~~It is implementation-defined whether over-aligned types are supported (3.11).~~ [*Note:* ...

Change 5.3.4p8:

> A *new-expression* may obtain storage for the object by calling an allocation function (3.7.4.1). If the *new-expression* terminates by throwing an exception, it may release storage by calling a deallocation function (3.7.4.2). If the allocated type is a non-array type, the allocation function's name is `operator new` and the deallocation function's name is `operator delete`. If the allocated type is an array type, the allocation function's name is `operator new[]` and the deallocation function's name is `operator delete[]`. [*Note:* an implementation shall provide default definitions for the global allocation functions (3.7.4, 18.6.1.1, 18.6.1.2). A C++ program can provide alternative definitions of these functions (17.6.4.6) and/or class-specific versions (12.5). The set of allocation and deallocation functions that may be called by a *new-expression* includes functions that are, semantically speaking, not allocation or deallocation functions; for example, see [new.delete.placement]. —*end note*]

Change 5.3.4p22:

> A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations (8.3.5), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds ~~the two-parameter form of~~ a usual deallocation function with a size parameter (3.7.4.2) and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed. For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function (5.3.5)

Change 5.3.5p5:

> If the object being deleted has incomplete class type at the point of deletion and the complete class is over-aligned or has a non-trivial destructor or a deallocation function, the behavior is undefined.

Consider whether to change 17.6.3.5p10:

> If the alignment associated with a specific over-aligned type is not supported by an allocator, instantiation of the allocator for that type may fail. The allocator also may silently ignore the requested alignment. [*Note:* Additionally, the member function `allocate` for that type may fail by throwing an object of type `std::bad_alloc`. —*end note*]

Even if the standard allocator is required to handle arbitrary alignments, it may be reasonable not to make the same requirement of user-defined allocators.

Change 17.6.4.6p2:

> A C++ program may provide the definition for any of ~~twelve~~ the following dynamic memory allocation function signatures declared in header `<new>` (3.7.4, 18.6):
>
> - `operator new(std::size_t)`
> - `operator new(std::size_t, const std::nothrow_t&)`

- `operator new[](std::size_t)`
- `operator new[](std::size_t, const std::nothrow_t&)`
- `operator delete(void*)`
- `operator delete(void*, const std::nothrow_t&)`
- `operator delete(void*, std::size_t)`
- `operator delete[](void*)`
- `operator delete[](void*, const std::nothrow_t&)`
- `operator delete[](void*, std::size_t)`

- operator new(std::size_t, std::align_val_t)
- operator new(std::size_t, std::align_val_t, const std::nothrow_t&)
- operator new[](std::size_t, std::align_val_t)
- operator new[](std::size_t, std::align_val_t, const std::nothrow_t&)
- operator delete(void*, std::align_val_t)
- operator delete(void*, std::align_val_t, const std::nothrow_t&)
- operator delete(void*, std::align_val_t, std::size_t)
- operator delete[](void*, std::align_val_t)
- operator delete[](void*, std::align_val_t, const std::nothrow_t&)
- operator delete[](void*, std::align_val_t, std::size_t)

Change the title of section 18.6.1.3:

18.6.1.3 ~~Placement~~ Non-allocating forms **[new.delete.placement]**

Change 20.7.9.1p5:

Returns: A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. ~~It is implementation-defined whether over-aligned types are supported (3.11).~~

Change 20.7.9.1p6:

Remark: the storage is obtained by calling `::operator new`~~`(std::size_t)`~~ (18.6.1), but it is unspecified when or how often this function is called. The use of `hint` is unspecified, but intended as an aid to locality if an implementation so desires.

Change 20.7.9.1p10:

Remarks: Uses `::operator delete`~~`(void*, std::size_t)`~~ (18.6.1), but it is unspecified when this function is called.

Specifications of library allocation and deallocation functions

Change section 18.6.1.1 as follows. (Some technically irrelevant editorial improvements are included; in CWG, these would be called “en passant” or “drive-by” changes.)

18.6.1.1 Single-object forms

```
void* operator new(std::size_t size);
void* operator new(std::size_t size, std::align_val_t alignment);
```

1 *Effects:* The allocation ~~function~~ functions ([basic.stc.dynamic.allocation]) called by a *new-expression* ([expr.new]) to allocate `size` bytes of storage. The first form is called for a type with fundamental alignment, and allocates storage suitably aligned to represent any object of that size. The second form is called for an over-aligned type, and allocates storage with the specified alignment.

2 *Replaceable:* a C++ program may define ~~a function with this function signature that displaces~~ functions with either of these function signatures, and thereby displace the default ~~version~~ versions defined by the C++ standard library.

3 *Required behavior:* Return a non-null pointer to suitably aligned storage ([basic.stc.dynamic]), or else throw a `bad_alloc` exception. This requirement is binding on ~~a~~ any replacement ~~version~~ versions of ~~this function~~ these functions.

4 *Default behavior:*

- Executes a loop: Within the loop, the function first attempts to allocate the requested storage. Whether the

attempt involves a call to the Standard C library function `malloc` or `aligned_alloc` is unspecified.

- Returns a pointer to the allocated storage if the attempt is successful. Otherwise, if the current `new_handler` ([get.new.handler]) is a null pointer value, throws `bad_alloc`.
- Otherwise, the function calls the current `new_handler` function ([new.handler]). If the called function returns, the loop repeats.
- The loop terminates when an attempt to allocate the requested storage is successful or when a called `new_handler` function does not return.

```
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
void* operator new(std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

5 *Effects:* Same as above, except that ~~it is~~ these are called by a placement version of a *new-expression* when a C++ program prefers a null pointer result as an error indication, instead of a `bad_alloc` exception.

6 *Replaceable:* a C++ program may define a function with this function signature that displaces the default version defined by the C++ standard library. *[This should be changed similarly to paragraph 2.]*

7 *Required behavior:* Return a non-null pointer to suitably aligned storage ([basic.stc.dynamic]), or else return a null pointer. ~~This~~ Each of these nothrow ~~version~~ versions of `operator new` returns a pointer obtained as if acquired from the (possibly replaced) ordinary version. This requirement is binding on a replacement version of this function. *[The last sentence should be changed similarly to paragraph 3.]*

8 *Default behavior:* Calls `operator new(size)`, or respectively `operator new(size, alignment)`. If the call returns normally, returns the result of that call. Otherwise, returns a null pointer.

9 [*Example:*

```
T* p1 = new T; // throws bad_alloc if it fails
T* p2 = new(nothrow) T; // returns nullptr if it fails
```

—*end example*]

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::align_val_t alignment, std::size_t size) noexcept;
```

10 *Effects:* The deallocation function ([basic.stc.dynamic.deallocation]) called by a *delete-expression* to render the value of `ptr` invalid.

11 *Replaceable:* a C++ program may define ~~a function with signature `void operator delete(void* ptr) noexcept` that displaces~~ functions with any of these signatures, and thereby displace the default version(s) defined by the C++ standard library. If ~~this~~ a function ~~(~~without a `size` parameter~~)~~ is defined, the program should also define ~~`void operator delete(void* ptr, td::size_t size) noexcept`~~ the corresponding function with a `size` parameter. If ~~this~~ a function with a `size` parameter is defined, the program shall also define the corresponding version without the `size` parameter. [*Note:* The default behavior below may change in the future, which will require replacing both deallocation functions when replacing the allocation function. —*end note*]

12 *Requires:* `ptr` shall be a null pointer or its value shall ~~be a value returned~~ point to a block of memory allocated by an earlier call to ~~the~~ a (possibly replaced) `operator new`~~`(std::size_t)` or `operator new(std::size_t, const std::nothrow_t&)`~~ which has not been invalidated by an intervening call to `operator delete`~~`(void*)` or `operator delete(void*, std::size_t)`~~.

13 *Requires:* If an implementation has strict pointer safety ([basic.stc.dynamic.safety]) then `ptr` shall be a safely-derived pointer.

14 *Requires:* If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function not taking an alignment argument. If present, the `alignment` argument shall equal the alignment argument passed to the allocation function that returned `ptr`. If present, the ~~`std::size_t`~~ `size` argument shall equal the size argument passed to the allocation function that returned `ptr`.

15 *Required behavior:* ~~Calls~~ A call to an `operator delete`~~`(void* ptr, std::size_t size)`~~ taking a size may be changed to ~~calls~~ a call to the corresponding `operator delete`~~`(void* ptr)`~~ not taking a size, without affecting memory allocation. [*Note:* A conforming implementation is for `operator delete(void* ptr, std::size_t size)` to simply call `operator delete(ptr)`. —*end note*]

16 *Default behavior:* the function `operator delete(void* ptr, std::size_t size)` calls `operator delete(ptr)`. The function

`operator delete(void* ptr, std::align_val_t alignment, std::size_t)` calls `operator delete(ptr, alignment).` [*Note:* See the note in the above *Replaceable* paragraph. *—end note*]

17 ~~*Default*~~ *Required behavior:* If `ptr` is null, does nothing. Otherwise, reclaims the storage allocated by the earlier call to `operator new`.

18 *Remarks:* It is unspecified under what conditions part or all of such reclaimed storage will be allocated by subsequent calls to `operator new` or any of `aligned_alloc,` `calloc`, `malloc`, or `realloc`, declared in `<cstdlib>`.

```
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

19 *Effects:* The deallocation function ([basic.stc.dynamic.deallocation]) called by the implementation to render the value of `ptr` invalid when the constructor invoked from a nothrow placement version of the *new-expression* throws an exception.

20 *Replaceable:* a C++ program may define a function with ~~signature `void operator delete(void* ptr, const std::nothrow_t&) noexcept` that displaces~~ either of these signatures, and thereby displace the default version(s) defined by the C++ standard library.

21 *Requires:* If an implementation has strict pointer safety ([basic.stc.dynamic.safety]) then `ptr` shall be a safely-derived pointer.

22 *Default behavior:* The function `operator delete(void* ptr, const std::nothrow_t&)` calls `operator delete(ptr)`. The function `operator delete(void* ptr, std::align_val_t alignment, const std::nothrow_t&)` calls `operator delete(ptr, alignment).`

Change section 18.6.1.2:

18.6.1.2 Array forms

```
void* operator new[](std::size_t size);
void* operator new[](std::size_t size, std::align_val_t alignment);
```

1 *Effects:* The allocation ~~function~~ functions ([basic.stc.dynamic.allocation]) called by the array form of a *new-expression* ([expr.new]) to allocate `size` bytes of storage. The first form is called for a type with fundamental alignment, and allocates storage suitably aligned to represent any array object of that size or smaller. The second form is called for an over-aligned type, and allocates storage with the specifed alignment.

Footnote: It is not the direct responsibility of `operator new[](std::size_t)` or `operator delete[](void*)` to note the repetition count or element size of the array. Those operations are performed elsewhere in the array `new` and `delete` expressions. The array `new` expression, may, however, increase the `size` argument to `operator new[](std::size_t)` to obtain space to store supplemental information.

2 *Replaceable:* a C++ program ~~can~~ may define a function with this function signature that displaces the default version defined by the C++ standard library. *[This should be changed similarly to paragraph 2 above.]*

3 *Required behavior:* Same as for ~~`operator new(std::size_t)`~~ the corresponding single-object forms. This requirement is binding on a replacement version of this function. *[The last sentence should be changed similarly to paragraph 3 above.]*

4 *Default behavior:* Returns `operator new(size)`, or respectively `operator new(size, alignment)`.

```
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
void* operator new[](std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

5 *Effects:* Same as above, except that ~~it is~~ these are called by a placement version of a *new-expression* when a C++ program prefers a null pointer result as an error indication, instead of a `bad_alloc` exception.

6 *Replaceable:* a C++ program ~~can~~ may define a function with this function signature that displaces the default version defined by the C++ standard library. *[This should be changed similarly to paragraph 2.]*

7 *Required behavior:* Return a non-null pointer to suitably aligned storage ([basic.stc.dynamic]), or else return a null pointer. ~~This~~ Each of these nothrow ~~version~~ versions of `operator new[]` returns a pointer obtained as if acquired from the (possibly replaced) ~~`operator new[](std::size_t)` function~~ ordinary version. This requirement is binding on a replacement version of this function. *[The last sentence should be changed similarly to paragraph 3.]*

8 *Default behavior:* Calls `operator new[](size)`, or respectively `operator new[](size, alignment)`. If the call returns normally, returns the result of that call. Otherwise, returns a null pointer.

```
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment, std::size_t size) noexcept;
```

9 *Effects:* The deallocation function ([basic.stc.dynamic.deallocation]) called by the array form of a *delete-expression* to render the value of `ptr` invalid.

10 *Replaceable:* a C++ program ~~can~~ may define ~~a function with signature `void operator delete[](void* ptr) noexcept` that displaces~~ functions with any of these signatures, and thereby displace the default version(s) defined by the C++ standard library. If ~~this~~ a function ~~(~~without a `size` parameter~~)~~ is defined, the program should also define ~~`void operator delete[](void* ptr, std::size_t size) noexcept`~~ the corresponding function with a `size` parameter. If ~~this~~ a function with a `size` parameter is defined, the program shall also define the corresponding version without the `size` parameter. [*Note*: The default behavior below may change in the future, which will require replacing both deallocation functions when replacing the allocation function. —*end note*]

11 *Requires:* `ptr` shall be a null pointer or its value shall ~~be the value returned~~ point to a block of memory allocated by an earlier call to a (possibly replaced) `operator new[]`~~`(std::size_t)` or `operator new[](std::size_t, const std::nothrow_t&)`~~ which has not been invalidated by an intervening call to `operator delete[]`~~`(void*)`~~(~~or `operator delete[](void*, std::size_t)`~~.

12 *Requires:* If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function not taking an alignment argument. If present, the `alignment` argument shall equal the alignment argument passed to the allocation function that returned `ptr`. If present, the ~~`std::size_t`~~ `size` argument ~~must~~ shall equal the size argument passed to the allocation function that returned `ptr`.

13 *Required behavior:* ~~Calls~~ A call to `operator delete[]`~~`(void* ptr, std::size_t size)`~~ taking a size may be changed to ~~calls~~ a call to `operator delete[]`~~`(void* ptr)`~~ not taking a size, without affecting memory allocation. [*Note:* A conforming implementation is for `operator delete[](void* ptr, std::size_t size)` to simply call `operator delete[](void* ptr)`. —*end note*]

14 *Requires:* If an implementation has strict pointer safety ([basic.stc.dynamic.safety]) then `ptr` shall be a safely-derived pointer.

15 *Default behavior:* ~~`operator delete[](void* ptr, std::size_t size)` calls `operator delete[](ptr)`, and `operator delete[](void* ptr)` calls `operator delete(ptr)`.~~ The functions that have a `size` parameter call and forward their other parameters to the corresponding function without a `size` parameter. The functions that do not have a `size` parameter call and forward their parameters to the corresponding `operator new` (single-object) function.

[This is the one place in the standard where the default behavior needs to be specified for four replaceable functions. My senses of esthetics and propriety rebelled at the prospect of enumerating them all, especially in a single flowing paragraph.]

```
void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

1 *Effects:* The deallocation function ([basic.stc.dynamic.deallocation]) called by the implementation to render the value of `ptr` invalid when the constructor invoked from a nothrow placement version of the array *new-expression* throws an exception.

2 *Replaceable:* a C++ program may define a function with ~~signature `void operator delete[](void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept` that displaces~~ either of these signatures, and thereby displace the default version defined by the C++ standard library.

3 *Requires:* If an implementation has strict pointer safety ([basic.stc.dynamic.safety]) then `ptr` shall be a safely-derived pointer.

4 *Default behavior:* The function `operator delete[](void* ptr, const std::nothrow_t&)` calls `operator delete[](ptr)`. The function `operator delete[](void* ptr, std::align_val_t alignment, const std::nothrow_t&)` calls `operator delete[](ptr, alignment)`.